HATHOR-CORE · MASTER-BOOK

Wallets, Keys & Crypto

How ownership is held and proven — public/private keypairs, digital signatures, addresses, and the HD wallets that manage them all from a single seed.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	40 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Asymmetric crypto · Public/private keys · ECDSA · secp256k1 · Digital signatures · Addresses · hash160 · base58check · HD wallet · BIP32/BIP39 · KeyPair

Table of Contents

Chapter 40 — Wallets, Keys & Crypto

What you'll learn in this chapter

- Why **owning a coin** in a blockchain means being able to prove a secret without revealing it, and how a public/private **keypair** plus a **digital signature** make that possible.
- The conceptual machinery underneath: **asymmetric cryptography**, **ECDSA** on the **secp256k1** curve, and what a signature actually proves.
- The **address pipeline** — how a public key becomes the short string you paste into a wallet (`hash160` → `base58check`), traced through `hathor/crypto/util.py`.
- The two wallet kinds in `hathor-core`: the `KeyPair` / `Wallet` (a bag of individual encrypted keys) and the `HDWallet` (one **seed** grows an entire tree of keys via **BIP32/BIP39**).
- How a wallet **tracks its UTXOs**, **selects inputs**, and **signs** a transaction so the resulting `input.data` satisfies the P2PKH script from Chapter 31.
- Why this code leans on the `cryptography` library and `pycoin` instead of hand-rolling crypto — and why that is the only defensible choice.

This chapter sits at the boundary between the node and the user. Everything before it — the vertex model, storage, verification, consensus — is about a node faithfully recording and agreeing on a ledger. But a ledger is only useful if people can move value on it, and moving value means answering one hard question: **how do I prove that a particular coin is mine, to a network of strangers, none of whom I trust and who do not trust me?** This chapter is the answer. It is also the one place in the book where cryptography stops being a black box.

We build the idea up from nothing — assume you have never met a public key — then walk the real code.

40.1 The problem: proving ownership without a trusted referee

Start from where Chapter 7 left you.

> **Recap — the UTXO and "ownership = a satisfiable lock" (full treatment in Ch. 7).** In Hathor there are no account balances stored anywhere. Instead there are **UTXOs**: unspent transaction outputs. Each output carries a locking script — a small puzzle. To spend that output, a new transaction's input must supply data that satisfies the puzzle. So "owning 5 HTR" does not mean a row in a table says so; it means there exists an unspent output whose lock you, and only you, can open. **Ownership is the ability to satisfy a lock.** → full treatment in Ch. 7.

The locking script Hathor uses by default is **P2PKH** — "pay to public-key hash." Its puzzle, in words, is: "To spend me, show a public key that hashes to this fixed value, and show a valid signature over this transaction made with the matching private key." You met the machine that checks this puzzle in Chapter 31.

> **Recap — script evaluation and CHECKSIG (full treatment in Ch. 31).** Verification runs each output's locking script together with the spending input's data on a small stack machine. For P2PKH the decisive opcode is `OP_CHECKSIG`: it pops a public key and a signature off the stack and checks the signature against the transaction's bytes. If the check fails, the whole transaction is invalid. The script evaluator never trusts the spender's word — it re-runs the cryptographic check itself. → full treatment in Ch. 31.

So the node side of ownership is settled: verification will demand a valid signature. What is missing is the other side — the side that **produces** that signature, manages the secrets it needs, and knows which outputs belong to "me." That is the wallet. And the entire scheme rests on a single piece of mathematics that lets you prove you know a secret without ever showing it. We have to understand that first.

Here is the naive approach and why it fails. Imagine ownership were proved with a password: the output says "spendable by anyone who knows the word `swordfish`," and to spend it you publish `swordfish`. This is hopeless. The moment you broadcast the spending transaction, the whole network sees `swordfish`, and any one of them can race to spend the next output you protect the same way. A shared-secret password reveals itself in the act of using it.

What we need is a secret you can **prove you possess** without ever transmitting it. That is exactly what asymmetric cryptography provides.

40.2 A primer on asymmetric cryptography and digital signatures

This section teaches the concept with neutral, generic framing. The Hathor code comes after.

40.2.1 Two keys instead of one

In ordinary ("symmetric") cryptography there is one secret key, shared by both sides — the `swordfish` problem. **Asymmetric**[1] cryptography uses two keys that are mathematically linked:

- a **private key**[2] — a large secret number you keep and never reveal;
- a **public key**[3] — derived from the private key, which you can hand out freely.

The link is one-directional in a precise sense: it is easy to compute the public key from the private key, but — for the curve Hathor uses — infeasible to go backwards and recover the private key from the public key. The pair together is a **keypair**[4].

A useful mental picture: the private key is a unique physical seal-stamp locked in your desk; the public key is a photograph of the seal's imprint that you publish. Anyone can look at the photograph and recognise a genuine imprint; nobody can carve a matching stamp from the photo alone.

40.2.2 What a digital signature is

A **digital signature**[5] is a function of two things: a message (some bytes) and a private key. It produces a short blob of bytes — the signature — with two properties:

1. **Verifiable.** Anyone holding the matching public key, the message, and the signature can check that the signature was produced by the matching private key over exactly that message. This check is a `True` / `False` test; it needs nothing secret.
2. **Unforgeable.** Without the private key, you cannot produce a signature that passes the check — not for a new message, and not by tampering with the message of an existing valid signature (changing one byte of the message invalidates the signature).

Put those together and you have the answer to §40.1:

```
SIGNING  (done by the owner, with the SECRET key)
   message  +  private_key   ──sign──▶   signature

VERIFYING  (done by anyone, with the PUBLIC key)
   message  +  public_key  +  signature  ──verify──▶  True / False
```

The owner signs with the private key; everyone else verifies with the public key. The secret never travels. This is the cryptographic spelling of "ownership = the ability to satisfy a lock": the lock says "produce a signature that verifies against this public key," and only the holder of the matching private key can.

There is one subtlety worth naming now because it recurs in the code. You almost never sign a raw message directly; you sign a **hash**[6] of it. A hash function squeezes a message of any length into a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably. Signing the hash rather than the whole message keeps signatures a fixed size and is how every real signature scheme works. Hold this thought — you will see Hathor hash the transaction before signing it.

40.2.3 ECDSA on secp256k1 — the concrete scheme, conceptually

"Asymmetric crypto" is a family. The specific scheme Hathor uses — the same one Bitcoin uses — is **ECDSA**[7]: the Elliptic Curve Digital Signature Algorithm. You do **not** need the mathematics to read this chapter; you need three facts.

1. **It is built on an elliptic curve**, a particular kind of mathematical structure on which "multiplication" is easy one way and "division" is infeasible the other way. That one-way-ness is what makes private→public easy and public→private infeasible.
2. **The specific curve is named `secp256k1`.** A "curve" here is just a fixed set of public parameters that every participant agrees on, the same way everyone agrees on the genesis block. Hathor, Bitcoin, and Ethereum all use `secp256k1`. Because it is standard, keys and signatures are interoperable across tools.
3. **The private key is a 256-bit number** (hence "256" in the name) and the public key is a point on that curve, which can be encoded compactly.

That is the whole conceptual budget. When the code says `ec.SECP256K1()` it is naming this curve; when it says `ec.ECDSA(...)` it is naming this signature scheme. Everything else is plumbing.

40.3 Localization

Three small packages collaborate, in a layered way. The lowest layer wraps a third-party crypto library; the middle layer derives keys; the top layer is the wallets.

```
hathor-core/
└── hathor/
    ├── crypto/
    │   ├── __init__.py
    │   └── util.py              ← the crypto wrapper: key gen, address pipeline,    ◀ YOU ARE
    │                              hash160, base58check, pubkey (de)serialization
    │
    ├── pycoin/
    │   ├── __init__.py
    │   └── htr.py               ← registers the "HTR" network with the pycoin lib
    │                              (BIP32 prefixes + version bytes) for HD derivation
    │
    └── wallet/                  ← the wallet implementations
        ├── __init__.py          ← exports Wallet, KeyPair, BaseWallet, HDWallet
        ├── base_wallet.py       ← BaseWallet: UTXO tracking, input selection, signing
        ├── keypair.py           ← KeyPair: one individual, encrypted private key
        ├── wallet.py            ← Wallet: a file-backed bag of KeyPairs (the "keypair" walle
        ├── hd_wallet.py         ← HDWallet: one seed → a whole tree of keys (BIP32/BIP39)
        ├── exceptions.py        ← InsufficientFunds, PrivateKeyNotFound, WalletLocked, …
        └── … (resources/, util, etc.)

hathor_cli/
└── generate_valid_words.py     ← CLI tool: print a fresh BIP39 mnemonic (seed words)
```

The signature-producing code in P2PKH form lives one package over, in `hathor/transaction/scripts/p2pkh.py` (the `P2PKH` script class from Ch. 31). The wallet calls into it to assemble the `input.data` that satisfies the lock. We cite it where it is used.

Context. `hathor/crypto`, `hathor/pycoin`, and `hathor/wallet` are the node's key-management and signing surface. The verification pipeline (Ch. 31) demands valid signatures over outputs; this is the machinery that **produces** them, manages the secrets behind them, and tracks which outputs a given key controls. It is optional to a node's consensus role — a node can verify the whole ledger without holding any keys — but it is what lets a node, or a wallet built on the node, actually spend.

40.4 What it does and why it exists

A wallet, stripped to its job description, does four things:

1. **Holds secrets safely.** It stores private keys, ideally encrypted at rest, and hands out the matching public keys / addresses freely.
2. **Recognises incoming money.** When a new transaction lands, it checks each output: "is this output locked to one of my addresses?" If so, it is a UTXO the wallet now controls.

3. **Selects inputs to spend.** When you ask it to send N tokens, it picks enough of its own UTXOs to cover N (plus change), avoiding locked or already-pending ones.
4. **Signs.** It computes the bytes that must be signed (the **sighash**), signs them with the right private key for each chosen input, and packs `<signature> <public-key>` into the input's `data` field so the P2PKH lock will open.

Why does this live inside the full node at all, rather than only in a separate wallet app? Because `hathor-core` ships operator and developer tooling — generate a wallet, send a test transaction, run a faucet — and the node's HTTP API exposes a "send tokens" endpoint that needs to build and sign transactions server-side. A production node usually runs without an unlocked wallet; the wallet code is there for tooling, tests, and lightweight setups.

The package gives you **two flavours** of wallet, and the difference is entirely about where the keys come from:

- The `Wallet` (a.k.a. the "keypair" wallet) is a flat collection of independently-generated `KeyPair` objects, each an encrypted private key, persisted to a `keys.json` file. To back it up you must back up every key. (`hathor/wallet/wallet.py`.)
- The `HDWallet` ("hierarchical deterministic") generates every key, deterministically, from a single master **seed**, which is itself produced from a human-readable list of **mnemonic** words. To back it up you write down the words. One sheet of paper restores an unlimited key tree. (`hathor/wallet/hd_wallet.py`.)

Both share a large base class, `BaseWallet`, which holds all the UTXO-tracking, input-selection and signing logic that does not care where keys come from.

40.5 The concepts it rests on

Three earlier threads converge here. Rather than re-teach them, here are the pointers.

> **Recap — the `cryptography` dependency (foundations in Ch. 13).** Hathor does not implement elliptic-curve maths itself. It depends on `cryptography`, the standard, audited Python library backed by OpenSSL, declared in `pyproject.toml`. All low-level key generation, signing, and serialization in `hathor/crypto/util.py` are thin calls into `cryptography.hazmat.primitives.asymmetric.ec`. The "hazmat" in that import path is the library authors' own warning label — "hazardous materials" — flagging that these are low-level primitives to be wrapped carefully, which is exactly what `crypto/util.py` does. → dependency manifest in Ch. 13 / Appendix B.

> **Recap — TxInput / TxOutput and the address (full treatment in Ch. 25).** A transaction output is a `TxOutput` carrying a `value`, a `token_data` byte, and a `script` (the lock). A transaction input is a `TxInput` naming a previous output by `(tx_id, index)` and carrying a `data` field (the lock-opening proof). The **address** you paste into a wallet is not stored in the output directly — it is encoded inside the output's `script`. The wallet's job is to translate between "address" and "script," and to fill in each input's `data`. → full treatment in Ch. 25.

> **Recap — the UTXO set and the index (full treatment in Ch. 7 & 28).** "What can this wallet spend?" is answered by the set of unspent outputs locked to its keys. The node maintains UTXO/address indexes for fast lookup (Ch. 28), but the wallet also keeps its own in-memory bookkeeping (`unspent_txs`, `spent_txs`, ...) updated from pub-sub events, because a wallet must track its own outputs precisely, including ones in intermediate "maybe spent" states. → full treatment in Ch. 7 (the model) & Ch. 28 (the index).

40.6 The code, walked

We go bottom-up: first the address pipeline (pure crypto, no wallet), then the two key sources, then the tracking-and-signing logic that sits above both.

40.6.1 The address pipeline: public key → address

An **address** is a short, copy-pasteable, typo-resistant string that stands in for a public key. You never paste a raw public key; you paste an address derived from it. The derivation is a fixed pipeline, and it lives entirely in `hathor/crypto/util.py`.

Generically, the pipeline is:

```
public key
   │  (1) serialize to compact bytes
   ▼
public key bytes
   │  (2) hash160  =  ripemd160(sha256(bytes))      → 20 bytes
   ▼
public-key hash
   │  (3) prepend a 1-byte "version" tag
   ▼
version ‖ hash                                       → 21 bytes
   │  (4) append a 4-byte checksum = first 4 bytes of sha256(sha256(version‖hash))
   ▼
version ‖ hash ‖ checksum                            → 25 bytes  (raw address)
   │  (5) base58-encode the 25 bytes
   ▼
base58check address  (the string you paste)
```

Step (2) is **hash160**[9] — sha256 followed by ripemd160. Hashing the public key (rather than publishing it directly) shortens the address to 20 bytes and adds a layer of protection: the public key is only revealed when you spend, not when you receive. Step (4)'s checksum is what makes addresses typo-resistant — change one character and the recomputed checksum won't match, so a mistyped address is rejected before any coins move. The whole "version byte + payload + double-sha256 checksum, base58-encoded" envelope is called **base58check**[10].

Now the real code. Hash160 is defined with a fallback for older Python builds that lack `ripemd160` in `hashlib`, in which case it borrows pycoin's pure-Python implementation:

```python
# hathor/crypto/util.py:62
def get_hash160(public_key_bytes: bytes) -> bytes:
    """The input is hashed twice: first with SHA-256 and then with RIPEMD-160"""
    key_hash = hashlib.sha256(public_key_bytes)
    h = hashlib.new('ripemd160')
    h.update(key_hash.digest())
    return h.digest()
```

The public key is first serialized to compact bytes by `get_public_key_bytes_compressed` (`crypto/util.py:180`), which asks `cryptography` for the X9.62 compressed point encoding. The full address build is `get_address_from_public_key_hash`:

```python
# hathor/crypto/util.py:120
def get_address_from_public_key_hash(public_key_hash: bytes, version_byte: Optional[bytes] = None
    settings = get_global_settings()
    address = b''
    actual_version_byte: bytes = version_byte if version_byte is not None else settings.P2PKH_VER
    address += actual_version_byte        # (3) version tag
    address += public_key_hash            # the 20-byte hash160
    checksum = get_checksum(address)      # (4) checksum over version‖hash
    address += checksum
    return address
```

The checksum is the classic double-sha256, first four bytes (`crypto/util.py:144`):

```python
# hathor/crypto/util.py:144
def get_checksum(address_bytes: bytes) -> bytes:
    """ Calculate double sha256 of address and gets first 4 bytes """
    return hashlib.sha256(hashlib.sha256(address_bytes).digest()).digest()[:4]
```

The final base58 step is a one-liner wrapping the `base58` library — e.g. `get_address_b58_from_public_key_hash` (`crypto/util.py:107`) calls `base58.b58encode(...)`. The whole chain from a public-key object to a base58 string is `get_address_b58_from_public_key` (`crypto/util.py:96`).

Going the other way, `decode_address` (`crypto/util.py:241`) base58-decodes the string, asserts it is exactly 25 bytes, **re-computes the checksum and compares** it, and raises `InvalidAddress` on mismatch. This is the typo-catching guarantee in action: a corrupted address can essentially never pass the checksum by accident.

The **version byte** (step 3) is how the same pipeline distinguishes address types. `P2PKH_VERSION_BYTE` tags ordinary single-key addresses; `MULTISIG_VERSION_BYTE` tags multisig addresses (`crypto/util.py:202`, `get_address_b58_from_redeem_script_hash`), which hash a redeem script instead of a single public key. The version byte also differs by network (mainnet vs testnet), so a testnet address visibly differs from a mainnet one — a guard against sending real coins to a test address.

One detail to internalise: `secp256k1` is named explicitly when a public key is reconstructed from bytes:

```
# hathor/crypto/util.py:191
def get_public_key_from_bytes_compressed(public_key_bytes: bytes) -> ec.EllipticCurvePublicKey:
    return ec.EllipticCurvePublicKey.from_encoded_point(ec.SECP256K1(), public_key_bytes)
```

That `ec.SECP256K1()` is the concrete curve from §40.2.3, sitting in real code.

40.6.2 KeyPair — one individual, encrypted key

The smallest unit of key management is `KeyPair` (`hathor/wallet/keypair.py:26`). It is not a wallet; it is one private key, stored **encrypted**, plus the address it derives to:

```
# hathor/wallet/keypair.py:31
def __init__(self, private_key_bytes: Optional[bytes] = None, address: Optional[str] = None,
             used: bool = False) -> None:
    """Holds the address in base58 and the encrypted bytes of the private key"""
    self.private_key_bytes = private_key_bytes   # ENCRYPTED bytes, not the raw key
    self.address = address
    self.used = used
    self._cache_priv_key_unlock = None           # decrypted key, cached only after unlock
```

Creating a fresh key is where `secp256k1` and the `cryptography` library meet (`keypair.py:116`):

```
# hathor/wallet/keypair.py:123 (inside KeyPair.create)
new_key = ec.generate_private_key(ec.SECP256K1(), default_backend())
private_key_bytes = get_private_key_bytes(
    new_key, encryption_algorithm=serialization.BestAvailableEncryption(password))
address = get_address_b58_from_public_key(new_key.public_key())
```

Three things happen in those lines. A random `secp256k1` private key is generated. It is serialized to bytes encrypted under a password (`BestAvailableEncryption`) — so what `KeyPair` holds on disk is ciphertext, useless without the password. And the matching address is computed once via the §40.6.1 pipeline and stored alongside.

To use the key you must decrypt it, which requires the password (`keypair.py:61`). `get_private_key` decrypts on first call and caches the decrypted object in `_cache_priv_key_unlock`; `clear_cache` (`keypair.py:56`) wipes that cache when the wallet locks. A wrong password raises `IncorrectPassword`; no password at all raises `WalletLocked`. This is the at-rest-encryption story: the secret is only ever decrypted into memory while the wallet is explicitly unlocked.

`KeyPair` can also produce a P2PKH input proof on its own (`keypair.py:94`):

```python
# hathor/wallet/keypair.py:94
def p2pkh_create_input_data(self, password: bytes, data: bytes) -> bytes:
    """Return a script input to solve the p2pkh script generated by this key pair."""
    private_key = self.get_private_key(password)
    public_key = private_key.public_key()
    public_key_bytes = get_public_key_bytes_compressed(public_key)
    signature = private_key.sign(data, ec.ECDSA(hashes.SHA256()))   # ECDSA over SHA-256
    script_input = P2PKH.create_input_data(public_key_bytes, signature)
    return script_input
```

There is the full signature in miniature: decrypt the private key, derive the public key, **sign with ECDSA/SHA-256**, then hand both the public key and the signature to `P2PKH.create_input_data` to pack into the input. We will see this same shape, generalised across a whole transaction, in §40.6.5.

40.6.3 Wallet — a file-backed bag of KeyPairs

`Wallet` (`hathor/wallet/wallet.py:32`) is the "keypair" wallet: a dictionary of `KeyPair`s keyed by base58 address, persisted to `keys.json`. Its responsibilities beyond `BaseWallet` are local: generate new keys in batches (`generate_keys`, `wallet.py:173`), hand out unused addresses (`get_unused_address`, `wallet.py:148`), read/write the keys file (`read_keys_from_file`, `wallet.py:78`), and unlock/lock by holding or clearing the password (`wallet.py:116`, `wallet.py:141`).

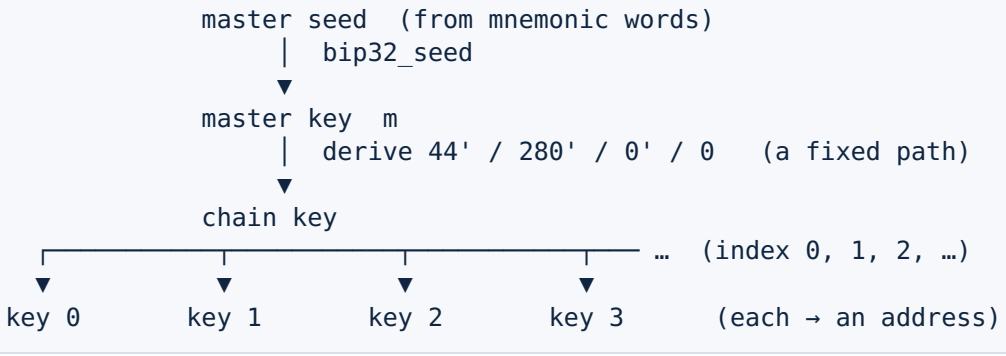
The `used` flag and the `unused_keys` set implement a privacy convention: each address is meant to be used once. When tokens arrive at an address, `tokens_received` (`wallet.py:194`) marks it used and removes it from `unused_keys`; when you need a fresh receive address, `get_unused_address` pops an unused one (generating a new batch if the wallet is unlocked and the pool is empty). Reusing addresses links your transactions together on the public ledger, so wallets cycle through fresh addresses by default.

Its signing helper is `get_input_aux_data` (`wallet.py:208`), which we look at in §40.6.5 alongside the HD wallet's.

40.6.4 HDWallet — one seed, a whole tree

The `Wallet` has a backup problem: every key is independent random data, so a complete backup means copying every key, and a backup taken yesterday misses keys generated today. **Hierarchical-deterministic** wallets solve this. The idea, generically:

> Start from one master secret (the **seed**). Define a deterministic function `derive(parent_key, index) → child_key`. Apply it repeatedly to grow a **tree** of keys, where the path from the root (e.g. `account 0 / chain 0 / key 5`) names each key. Because everything is derived from the one seed by a fixed function, **the seed alone regenerates the entire tree** — past, present, and future keys.



Two standards make this interoperable across every wallet vendor:

- **BIP32**[11] defines the derivation function and the tree (the `derive(parent, index)` above).
- **BIP39**[12] defines how a human-readable list of words — the **mnemonic**[13] — maps to the binary seed. Twelve to twenty-four common English words are your master secret, in a form you can write on paper and read back without transcription errors.

Hathor's `HDWallet` (`hathor/wallet/hd_wallet.py:54`) implements exactly this, leaning on two libraries: the `mnemonic` package for BIP39, and `pycoin` for BIP32 derivation. The seed-to-tree construction is in `_manually_initialize` (`hd_wallet.py:122`):

```python
# hathor/wallet/hd_wallet.py:126 (inside _manually_initialize)
self.mnemonic = Mnemonic(self.language)
...
seed = self.mnemonic.to_seed(self.words, self.passphrase.decode('utf-8'))   # BIP39: words → seed

from pycoin.networks.registry import network_for_netcode
_register_pycoin_networks()
network = network_for_netcode('htr')
key = network.keys.bip32_seed(seed)                                         # BIP32: seed → maste

# Chain path = 44'/280'/0'/0
self.chain_key = key.subkey_for_path('44H/280H/0H/0')
for key in self.chain_key.children(self.initial_key_generation, 0, False):
    self._key_generated(key, key.child_index())
```

Read the derivation path `44H/280H/0H/0` — the `H` means "hardened," a BIP32 detail that strengthens a derivation step. The comment in the source spells it out: `44'` is the **BIP44** purpose tag, `280'` is the **coin type** that uniquely identifies Hathor (Bitcoin is `0`, Hathor is `280`), `0'` is the account number, and the final `0` is the external chain. So

Hathor keys live at a well-defined, interoperable location in the BIP44 hierarchy — another wallet that knows "coin type 280" can restore a Hathor HDWallet from the same words.

The `htr` network code is what `hathor/pycoin/htr.py` registers — this is the entire purpose of the `pycoin` package:

```
# hathor/pycoin/htr.py:21
network = create_bitcoinish_network(
    symbol='HTR', network_name='Hathor', subnet_name='mainnet',
    wif_prefix_hex='80',
    address_prefix_hex=settings.P2PKH_VERSION_BYTE.hex(),
    pay_to_script_prefix_hex=settings.MULTISIG_VERSION_BYTE.hex(),
    bip32_prv_prefix_hex='0488ade4', bip32_pub_prefix_hex='0488B21E',
)
```

`pycoin` natively understands Bitcoin-family networks; this call teaches it Hathor's version bytes (the same `P2PKH_VERSION_BYTE` from §40.6.1) so the addresses it derives match Hathor's address format. `_register_pycoin_networks` (`hd_wallet.py:38`) wires this module into pycoin's network registry via the `PYCOIN_NETWORK_PATHS` environment variable.

Note one consequence, flagged in the source by a `TODO` (`hd_wallet.py:29`): pycoin's BIP32 keys carry their own ECDSA implementation, separate from OpenSSL-backed `cryptography`. So the **HD wallet signs through pycoin**, while the **keypair `Wallet` signs through `cryptography`**. The two wallet kinds reach the same `secp256k1`/ECDSA result by different libraries — visible in the next section.

The **gap limit** (`hd_wallet.py:103`, default 20) is the HD-wallet rule that bounds how many unused addresses to scan ahead. Because keys are generated lazily, a wallet recovered from a seed must generate addresses and check each for history; the gap limit says "if you hit 20 consecutive empty addresses, stop — assume the rest are empty too." `tokens_received` (`hd_wallet.py:205`) keeps the buffer of pre-generated keys topped up so syncing never runs out of look-ahead addresses.

40.6.5 Tracking UTXOs, selecting inputs, and signing — the shared `BaseWallet`

Everything above is about where keys come from. The logic that uses the keys — recognise money, pick inputs, sign — is in `BaseWallet` (`hathor/wallet/base_wallet.py:71`) and is shared by both wallet kinds.

Tracking. The wallet keeps several dictionaries of its outputs: `unspent_txs`, `maybe_spent_txs`, `spent_txs`, plus voided variants (`base_wallet.py:107`–120). It learns about new transactions through pub-sub.

> **Recap — pub-sub (full treatment in Ch. 30).** Components announce events ("a new transaction is being processed," "a transaction's consensus status changed") on a `PubSubManager`; interested components subscribe to event types and get called back. The wallet is a subscriber. → full treatment in Ch. 30.

There are two subscription paths, and it pays to keep them distinct. The **manager** subscribes the wallet to `NETWORK_NEW_TX_PROCESSING`, routing each new vertex to `wallet.on_new_tx` (`hathor/manager.py:261`). Separately, `BaseWallet.start` subscribes to `CONSENSUS_TX_UPDATE` and `CONSENSUS_TX_REMOVED`, routing to `on_tx_update` (`base_wallet.py:133`, `base_wallet.py:182`). The first path notices new outputs and spends; the second reacts to consensus changes — a transaction becoming voided or winning a conflict — so the wallet can move outputs between its spent/unspent/voided buckets.

`on_new_tx` (`base_wallet.py:527`) is the recognise-money routine. For each output it parses the locking script and asks "is this address one of mine?" (`script_type_out.address not in self.keys`). If yes, it records a new `UnspentTx` and fires `WALLET_OUTPUT_RECEIVED`. For each input, it checks whether the spent output was one of ours and, if so, moves it to `spent_txs` and fires `WALLET_INPUT_SPENT`. Voided transactions are skipped (`base_wallet.py:535`).

Selecting inputs. When you want to send an amount, `get_inputs_from_amount` (`base_wallet.py:469`) walks the wallet's unspent outputs for the right token and accumulates them until they cover the requested amount, skipping outputs that are time-locked, are token authorities, or are freshly-mined rewards still within the reward-lock window (`can_spend_block`, `base_wallet.py:519`). The source is candid about the algorithm (`base_wallet.py:475`): "This is a very simple algorithm, so it does not try to find the best combination of inputs" — it is first-fit, not optimal coin-selection. If the unspent set can't cover the amount it raises `InsufficientFunds`. Chosen UTXOs are moved into `maybe_spent_txs` immediately so a second concurrent send can't pick them again. Change is handled by `handle_change_tx` (`base_wallet.py:444`): if the inputs over-cover the amount, it adds an output back to a fresh wallet address for the difference.

Signing. This is the payoff — where everything in §40.2 becomes bytes. The end-to-end builder is `prepare_transaction` (`base_wallet.py:214`):

```
# hathor/wallet/base_wallet.py:255 (inside prepare_transaction)
tx = cls(inputs=tx_inputs, outputs=tx_outputs, tokens=tokens, timestamp=timestamp)
data_to_sign = tx.get_sighash_all()

for txin, privkey in zip(tx.inputs, private_keys):
    public_key_bytes, signature = self.get_input_aux_data(data_to_sign, privkey)
    txin.data = P2PKH.create_input_data(public_key_bytes, signature)
```

Three steps, each worth naming:

1. **Compute the sighash.** `tx.get_sighash_all()` (`hathor/transaction/transaction.py:231`) serializes the transaction's inputs, outputs, and token list into bytes — the bytes that the signature commits to. The **sighash**[15] ("signature hash") is what you sign: by signing it, you sign the entire shape of the transaction, so nobody can redirect an output or change an amount without invalidating your signature. (The method excludes per-input data and caches its result, since it is the same for every input — `transaction.py:237`.)

2. **Sign, per input.** `get_input_aux_data` is the abstract hook (`base_wallet.py:211`) each wallet kind implements. For the keypair `Wallet` (`hathor/wallet/wallet.py:208`):

```
# hathor/wallet/wallet.py:220
public_key_bytes = get_public_key_bytes_compressed(private_key.public_key())
hashed_data = hashlib.sha256(data_to_sign).digest()
signature = private_key.sign(hashed_data, ec.ECDSA(hashes.SHA256()))
return public_key_bytes, signature
```

For the `HDWallet`, signing goes through pycoin instead (`hathor/wallet/hd_wallet.py:320`):

```
# hathor/wallet/hd_wallet.py:332
prehashed_msg = hashlib.sha256(hashlib.sha256(data_to_sign).digest()).digest()
signature = private_key.sign(prehashed_msg)
return private_key.sec(), signature
```

Both return `(public_key_bytes, signature)` — the public key in compressed form, and an ECDSA signature over the sighash. (The exact hashing differs by library: pycoin's `sign` expects an already-hashed message, so the HD path hashes explicitly; `cryptography`'s `ECDSA(SHA256)` hashes internally.)

1. **Pack the input data.** `P2PKH.create_input_data(public_key_bytes, signature)` (`hathor/transaction/scripts/p2pkh.py:94`) assembles the input's `data` field by pushing the signature and then the public key onto a script:

```
# hathor/transaction/scripts/p2pkh.py:94
@classmethod
def create_input_data(cls, public_key_bytes: bytes, signature: bytes) -> bytes:
    s = HathorScript()
    s.pushData(signature)
    s.pushData(public_key_bytes)
    return s.data
```

This closes the loop with Chapter 31. The locking script that `P2PKH.create_output_script` (`p2pkh.py:70`) put on the output was `OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`. The input data we just built is `<signature> <publicKey>`. When verification (Ch. 31) runs them together on the stack machine, `OP_HASH160` plus `OP_EQUALVERIFY` confirm the public key hashes to the address baked into the output, and `OP_CHECKSIG` confirms the signature verifies against that public key over the sighash. The lock opens **only** for the holder of the matching private key — the whole point of §40.2, now mechanical.

For signing an already-built transaction (rather than building one from scratch), `sign_transaction` (`base_wallet.py:425`) does the same per-input loop, but uses `match_inputs` (`base_wallet.py:940`) to sign only those inputs that spend the wallet's own outputs — leaving other parties' inputs untouched, which is what makes collaborative/multisig transactions possible.

40.6.6 Multisig, briefly

The same address machinery extends to **multisig** (multiple signatures). A multisig output locks to the hash of a redeem script of the form `<M> <pubkey1> … <pubkeyN> <N> OP_CHECKMULTISIG` — "any M of these N keys must sign." The address is built by `get_address_b58_from_redeem_script_hash` (`crypto/util.py:202`) using `MULTISIG_VERSION_BYTE`, and the script lives in `hathor/transaction/scripts/multi_sig.py`. Its `create_input_data` (`multi_sig.py:114`) packs the M signatures plus the redeem script, mirroring P2PKH's `<sig> <pubkey>` but with several signatures. The conceptual leap is small once P2PKH is understood: replace "one key must sign" with "M of N keys must sign," and reuse the same sighash, the same ECDSA, the same base58check envelope.

40.6.7 The CLI seed-words generator

The mnemonic that seeds an `HDWallet` has to come from somewhere safe. `hathor_cli/generate_valid_words.py` is the operator tool that prints a fresh BIP39 mnemonic:

```
# hathor_cli/generate_valid_words.py:18
def generate_words(language: str = 'english', count: int = 24) -> str:
    from mnemonic import Mnemonic
    mnemonic = Mnemonic(language)
    return mnemonic.generate(strength=int(count * 10.67))
```

The `count * 10.67` converts a word count (12–24) into the BIP39 entropy strength in bits — the same conversion `HDWallet.unlock` uses when it auto-generates words (`hd_wallet.py:295`). Run this command, write the words down, and you hold the master secret for an entire key tree.

40.7 Why these libraries, not hand-rolled crypto

This is a Track-C trade-off, and for cryptography the trade-off is unusually one-sided.

The first rule of cryptographic engineering is: do not implement your own. Not because it is hard to make code that produces a signature — that part is a few lines — but because it is brutally hard to make code that produces a signature without leaking the private key through a side channel. A naive ECDSA implementation can leak bits of the secret through how long it takes to run, through which memory it touches, or through subtle math errors that only bite on rare inputs. These bugs do not show up in tests; they show up as stolen funds. The industry's hard-won standard libraries have been audited for exactly these failure modes over years. So `hathor/crypto/util.py` is, deliberately, a thin wrapper over `cryptography` (OpenSSL-backed). It contributes Hathor-specific composition — the address pipeline, the version bytes, the encoding — and delegates every raw primitive (key generation, ECDSA sign/verify, point serialization) to the audited library.

Why `cryptography`? It is the de-facto standard asymmetric-crypto library in the Python ecosystem, backed by OpenSSL, widely audited, and exposing exactly the `secp256k1`/ECDSA primitives Hathor needs.

Why `pycoin` on top, for the HD wallet? Because **BIP32/BIP39 are standards, and the value of a standard is interoperability.** A user must be able to take their 24 words to any compliant wallet and recover the same keys. Re-implementing BIP32 derivation by hand would risk a subtle incompatibility (a wrong hardened-derivation step, a different seed-stretching) that would silently produce different keys — a backup that restores the wrong wallet is worse than no backup. `pycoin` is a mature, Bitcoin-family library that implements these BIPs; `hathor/pycoin/htr.py` only teaches it Hathor's version bytes. The cost is the seam noted in §40.6.4: pycoin carries its own ECDSA (the `TODO` at `hd_wallet.py:29` flags this as worth a security review), so the HD path does not go through OpenSSL. That is the price of standards-compliant derivation, accepted knowingly and marked in the code.

Why `mnemonic`? Same reasoning: BIP39's word list and seed-derivation are a standard, and the `mnemonic` package is its canonical Python implementation. Interoperability of the recovery words is the whole point.

The pattern across all three is identical: **own the composition, delegate the cryptography.** That is the correct posture for any application that touches keys.

40.8 How it plugs into the node's lifecycle

The wallet is an optional collaborator the builder may attach to the `HathorManager`. When present, the wiring is small and follows the lifecycle you met in Chapter 29.

> **Recap — manager start/stop (full treatment in Ch. 29).** The `HathorManager` is the central coordinator; `start()` brings subsystems online in order and `stop()` tears them down. Collaborators like the wallet are injected at construction and started/stopped by the manager. → full treatment in Ch. 29.

- **Injection.** At construction the manager gives the wallet its `pubsub` and `reactor` and subscribes it (`hathor/manager.py:221`–225). `_subscribe_wallet` (`manager.py:255`) routes `NETWORK_NEW_TX_PROCESSING` events to `wallet.on_new_tx`, so every accepted vertex passes under the wallet's eye.
- **Start.** Inside `manager.start()`, `wallet.start()` is called (`manager.py:334`), which subscribes the wallet to the consensus-update events (Ch. 30) and schedules the periodic `maybe_spent` cleanup timer (`base_wallet.py:152`).
- **Initialize.** During initialization the manager calls `wallet._manually_initialize()` (`manager.py:448`) — for a `Wallet` this loads `keys.json`; for an `HDWallet` it derives the seed and the first batch of keys.
- **Stop.** On shutdown `wallet.stop()` unsubscribes it from pub-sub (`manager.py:369`, `base_wallet.py:161`).
- **Signing in service of clients.** When the node's HTTP "send tokens" path builds a transaction, it asks the wallet for an unused address and to sign — the same `prepare_transaction` / `sign_transaction` flow from §40.6.5. The signed transaction then re-enters the ordinary ingestion pipeline (Ch. 33): it is **verified** (Ch. 31), where the very signature the wallet produced is checked by `OP_CHECKSIG`, then run through **consensus** (Ch. 32) and stored. The wallet produces proofs; verification consumes them. Producer and consumer, finally joined.

Recap

Concern	Where it lives	Central type / function
Asymmetric keypair + ECDSA on secp256k1	`cryptography` lib, called from `hathor/crypto/util.py`	`ec.SECP256K1()`, `ec.ECDSA(...)`
Address pipeline (hash160 → base58check)	`hathor/crypto/util.py:62`, `:120`, `:144`	`get_hash160`, `get_address_from_public_key_hash`, `get_checksum`
One encrypted individual key	`hathor/wallet/keypair.py:26`	`KeyPair` (`create:116`, `get_private_key:61`)

Concern	Where it lives	Central type / function
File-backed bag of keys	`hathor/wallet/wallet.py:32`	`Wallet`
Seed → tree of keys (BIP32/BIP39)	`hathor/wallet/hd_wallet.py:54`	`HDWallet` (`_manually_initialize:122`)
BIP32 network registration	`hathor/pycoin/htr.py:21`	`create_bitcoinish_network`
BIP39 mnemonic CLI	`hathor_cli/generate_valid_words.py:18`	`generate_words`
UTXO tracking via pub-sub	`hathor/wallet/base_wallet.py:527`	`BaseWallet.on_new_tx`
Input selection (first-fit)	`hathor/wallet/base_wallet.py:469`	`get_inputs_from_amount`
The sighash (bytes to sign)	`hathor/transaction/transaction.py:231`	`get_sighash_all`
Sign per input	`base_wallet.py:211` ; impls `wallet.py:208` , `hd_wallet.py:320`	`get_input_aux_data`
Pack `<sig> <pubkey>` into the input	`hathor/transaction/scripts/p2pkh.py:94`	`P2PKH.create_input_data`
Manager wiring	`hathor/manager.py:221` , `:255` , `:334` , `:448`	`_subscribe_wallet` , start/stop

Ownership in a blockchain is not a fact recorded in a table; it is the ability to produce a proof. This chapter showed the full circle: a private key is a secret number on the `secp256k1` curve; its public key derives, via `hash160` and `base58check`, into the address that locks an output; spending that output means signing the transaction's sighash with the matching private key and packing `<signature> <publicKey>` where the lock can check it. The two wallet kinds differ only in where keys come from — a flat encrypted bag (`Wallet`) or a single seed grown into a tree (`HDWallet`) — and both delegate the cryptography to audited libraries, owning only the composition. The wallet produces the proofs that verification (Ch. 31) demands.

The next chapter steps away from the ledger and into operations: **Chapter 41 — Runtime Control (`hathor/sysctl`)**, the control socket that lets an operator query and tune a running node without restarting it.

---

1. **Asymmetric cryptography** (also "public-key cryptography") uses a pair of mathematically linked keys — one private, one public — instead of a single shared secret. What one key does (e.g. sign), only the other can verify; you cannot derive the private key from the public key. ↩
2. A **private key** is a large secret number that you never reveal. Possessing it is what defines ownership; anyone who learns it can spend your coins. In Hathor it is a 256-bit `secp256k1` key, stored encrypted at rest. ↩
3. A **public key** is derived from the private key and may be shared freely. It is used to verify signatures and to derive your address. Computing it from the private key is easy; the reverse is infeasible. ↩
4. A **keypair** is the matched (private, public) pair generated together. The two are useless apart: the private key signs, the public key verifies, and only a matched pair agrees. ↩
5. A **digital signature** is a blob of bytes produced from a message and a private key. Anyone with the message, the signature, and the matching public key can confirm the signer held the private key and that the message is unaltered — without the signer revealing the key. ↩
6. A **hash function** maps a message of any size to a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably and you cannot run it backwards. Hathor uses SHA-256 and RIPEMD-160. Signatures are computed over a hash of the message, not the raw message. ↩
7. **ECDSA** = Elliptic Curve Digital Signature Algorithm. The specific signature scheme Hathor (and Bitcoin) use. "Elliptic curve" is the kind of one-way maths that makes private→public easy and the reverse infeasible. You can read this chapter without the maths. ↩
8. **secp256k1** is the name of the specific elliptic curve — a fixed, public set of parameters — that Hathor, Bitcoin, and Ethereum all use. Because it is a shared standard, keys and signatures are interoperable across tools. ↩
9. **hash160** is SHA-256 followed by RIPEMD-160, producing a 20-byte fingerprint of a public key. Hashing the public key shortens the address and hides the public key until the output is spent. ↩
10. **base58check** is an encoding for addresses: a version byte, then the payload, then a 4-byte checksum (`sha256(sha256(...))` truncated), all encoded in base58 (digits/letters minus look-alikes like 0/O, I/l). The checksum makes mistyped addresses detectable. ↩
11. **BIP32** is the standard defining hierarchical-deterministic key derivation: a function that grows a tree of child keys from one master key, so a single master secret regenerates every key. ("BIP" = Bitcoin Improvement Proposal.) ↩
12. **BIP39** is the standard mapping a list of memorable words (the mnemonic) to the binary seed that BIP32 starts from. It is why a wallet backup can be 24 words on paper instead of raw bytes. ↩
13. A **mnemonic** here is the ordered list of BIP39 words (12–24 of them) that encodes a wallet's seed. Anyone with the words can regenerate the entire key tree, so the words must be kept as secret as a private key. ↩
14. A **seed** is the master secret (a block of bytes) from which an HD wallet derives all its keys. It is produced from the BIP39 mnemonic (plus an optional passphrase) and fed into BIP32 derivation. ↩
15. The **sighash** ("signature hash") is the canonical serialization of a transaction's inputs, outputs, and tokens — the exact bytes a signature commits to. Signing the sighash means signing the whole shape of the transaction, so it cannot be altered after signing without breaking the signature. ↩